

UNIVERSIDAD NACIONAL AUTÓNOMA
DE MÉXICO

FACULTAD DE CIENCIAS

MODELADO Y PROGRAMACIÓN

DOCUMENTACIÓN

PRÁCTICA 4:
PRUEBAS UNITARIAS

DISEÑO DE PRUEBAS PARA UN SISTEMA

P R E S E N T A N :

INTEGRANTE DEL EQUIPO:	NÚM. DE CUENTA:
Alan Gael Alvarez Hernández	323113340
María Fernanda Miranda Sánchez	420102320
Pedro Pablo Ruiz Hernández	323198507

PROFESORA:
Karla Adriana Esquivel Guzmán

AYUDANTE DE TEORÍA:
Ossiel Aguilar Espíndola

AYUDANTE DE LABORATORIO:
Enoc Pineda Aguirre



FACULTAD DE CIENCIAS
UNAM

SEMESTRE 1-2027

Ciudad de México, 20 de octubre de 2026

Índice

1. Implementación Original del Sistema **3**

2. Reglas del Sistema: Ticket de Estacionamiento **4**

3. Construcción de la Tabla de casos **4**

3.1. Planteamiento de pruebas 4

3.2. Tabla de casos 5

4. Evidencia de Ejecución **7**

4.1. Casos Normales: 1, 2 y 3 7

4.2. Casos Frontera: 4, 5, 6, 7, 8 y 9 8

4.3. Casos Entrada Inválida: 10, 11 y 15 9

4.4. Casos Interacción entre reglas: 12 y 13 11

4.5. Caso Números decimales en el resultado: 14 13

5. Preguntas de análisis **14**

Implementación Original del Sistema

Dado el modelo del sistema de estacionamiento, se plantearán las pruebas necesarias para verificar que el comportamiento de la implementación original del sistema sea el esperado por el modelo del mismo.

En caso de que la implementación original del sistema no pase las pruebas planteadas, modificaremos el código para que este último tenga el comportamiento esperado por el modelo del sistema.

```
...

TARIFA_BOLETO_PERDIDO = 300.0

def calcular_total(self, minutos: int, tipo_cliente: str = "normal",
                   boleto_perdido: bool = False) -> float:
    """
    Calcula el total a pagar de acuerdo con el modelo de la práctica.

    Nota para el alumno:
    trate esta implementación como un sistema que debe verificarse.
    No asuma que todo lo que hace el código es correcto.
    """

    if minutos < 0:
        raise ValueError("Los minutos no pueden ser negativos")


    if tipo_cliente not in ("normal", "frecuente"):
        raise ValueError("Tipo de cliente no válido")


    if boleto_perdido:
        total = self.TARIFA_BOLETO_PERDIDO


    elif minutos < 15:
        total = 0.0


    elif minutos <= 60:
        total = 20.0


    else:
        horas_adicionales = (minutos - 60) // 60
        total = 20.0 + (horas_adicionales * 15.0)


    if tipo_cliente == "frecuente":
        total *= 0.90

    return round(total, 2)

...
```

Reglas del Sistema: Ticket de Estacionamiento

Las reglas del Sistema de Ticket de Estacionamiento son:

R1. Los minutos deben ser un número entero mayor o igual que 0.

R2. Los tipos de clientes válidos son:

- **"normal"**
- **"frecuente"**

R3. Si el boleto está perdido, el costo final es de $300.00, sin importar los minutos ni el tipo de cliente.

R4. Si los minutos son 0, el costo es $0.00

R5. De 1 a 15 minutos, el costo es $0.00

R6. De 16 a 60 minutos, el costo es de $20.00

R7. Después de 60 minutos se cobran $20.00 por la primera hora y $15.00 por cada hora adicional **iniciada**.

R8. Un cliente **"frecuente"** recibe un 10 % de descuento sobre el costo calculado, excepto cuando perdió el boleto.

R9. El costo nunca puede ser negativo.

R10. Si los minutos son negativos, el sistema debe lanzar un error.

R11. Si el tipo de cliente no es válido, el sistema debe lanzar un error.

Construcción de la Tabla de casos

Planteamiento de pruebas

Planteamos para el Sistema de Ticket de Estacionamiento:

- 3 casos normales;
- 6 casos frontera;
- 2 entradas inválidas;
- 2 casos de interacción entre reglas;
- 1 caso para números decimales en el resultado;
- 1 caso adicional propuesto por el equipo (entrada inválida).

Tabla de casos

Núm.	Caso	Entrada	Resultado esperado	Regla(s)	Justificación
1	Normal	10 minutos "normal" con boleto	$0.00	R5	Analiza el comportamiento del sistema entre 0 a 15 minutos.
2	Normal	40 minutos "normal" con boleto	$20.00	R6	Analiza el comportamiento del sistema entre 16 a 60 minutos.
3	Normal	1250 minutos "normal" con boleto	$320.00	R7	Analiza el comportamiento del sistema entre horas completas e inciadas.
4	Frontera	15 minutos "normal" con boleto	$0.00	R5	Analiza el comportamiento del sistema en el minuto 15, donde el costo permanece en $0.00 antes de cambiar en el minuto 16 a $20.00
5	Frontera	16 minutos "normal" con boleto	$20.00	R6	Analiza el comportamiento del sistema en el minuto 16, donde el costo cambia a $20.00 después del minuto 15, donde estaba a $0.00
6	Frontera	60 minutos "normal" con boleto	$20.00	R6	Analiza el comportamiento del sistema en el minuto 60, donde el costo permanece en $20.00 antes de cambiar en el minuto 61 a $35.00 por las horas iniciadas.
7	Frontera	61 minutos "normal" con boleto	$35.00	R7	Analiza el comportamiento del sistema en el minuto 61, donde el costo cambia a $35.00 por las horas inciadas después del minuto 60, donde estaba a $20.00
8	Frontera	120 minutos "normal" con boleto	$35.00	R7	Analiza el comportamiento del sistema en el minuto 120, donde el costo permanece en $35.00 por las horas completas antes de cambiar en el minuto 121 a $50.00 por las horas inciadas.

Núm.	**Caso**	**Entrada**	**Resultado esperado**	**Regla(s)**	**Justificación**
9	Frontera	121 minutos "normal" con boleto	$50.00	R7	Analiza el comportamiento del sistema en el minuto 121, donde el costo cambia a $50.00 por las horas iniciadas después del minuto 120, donde estaba a $35.00 por las horas completas.
10	Entrada inválida	-1 minuto "normal" con boleto	Lanza Excepción: Minutos inválidos.	R1, R9 y R10	Analiza el comportamiento del sistema cuando recibe numeros negativos, donde debería rechazarlos.
11	Entrada inválida	5 minutos "premium" con boleto	Lanza Excepción: Tipo de cliente inválido.	R2 y R11	Analiza el comportamiento del sistema cuando recibe tipos de clientes inexistentes, donde debería rechazarlos.
12	Interacción entre reglas	1250 minutos "frecuente" con boleto	$288.00	R7 y R8	Analiza el comportamiento del sistema cuando un cliente "frecuente" con boleto aplica su descuento del 10 % dados los minutos transcurridos.
13	Interacción entre reglas	1250 minutos "frecuente" sin boleto	$300.00	R3 y R8	Analiza el comportamiento del sistema cuando un cliente "frecuente" no tiene boleto, en cuyo no aplica su decuento del 10 % y se le cobra $300.00.
14	Números decimales en el resultado	61 minutos "frecuente" con boleto	$31.50	R7 y R8	Analiza el comportamiento del sistema cuando se le cobra al cliente "frecuente" con boleto por 61 minutos transcurridos, en cuyo caso debe cobrar lo justo, que es $31.50.
15	Entrada inválida	15.5 minutos "normal" con boleto	Lanza Excepción: Minutos inválidos.	R1	Analiza el comportamiento del sistema cuando recibe minutos en decimales, donde debería rechazarlos.

Tabla 1: Casos establecidos para el Sistema Ticket de Estacionamiento.

Evidencia de Ejecución

Casos Normales: 1, 2 y 3

Implementación de la Prueba en Python

Prueba encargada de verificar el comportamiento del sistema en los casos normales 1, 2 y 3.

```
    # Arrange
    @pytest.mark.parametrize(
        "minutos, esperado",
        [
                              # | Caso | Descripción                                  |
            (10, 0.00),       # |  1   | Cliente "normal" con boleto pasó 10 min.   |
            (40, 20.00),      # |  2   | Cliente "normal" con boleto pasó 40 min.   |
            (1250, 320.00), # |  3   | Cliente "normal" con boleto pasó 1250 min. |
        ],
    )
    def test_cliente_Normal_con_Boleto(sistema, minutos, esperado):
        """
        Verifica el cobro calculado para un cliente "normal" con boleto
        dado un cierto tiempo en minutos.
        """

        # Act
        resultado = sistema.calcular_total(minutos, "normal", False)

        # Assert
        assert resultado == esperado
```

Resultado de la Prueba

Los casos 1 y 2 pasaron la prueba; sin embargo el caso 3 la falló. Dado que el planteamiento de las pruebas esta basado en una interpretación correcta del modelo, analizamos el Código original.

Código Original

```
    ...
    else:
        horas_adicionales = (minutos - 60) // 60
        total = 20.0 + (horas_adicionales * 15.0)
    ...
```

Al revisar el Código original, notemos que el cálculo realizado para conocer el cobro del estacionamiento con horas adicionales es incorrecto, ya que se pierde información de las horas iniciadas si solo se toma la parte entera de la división entre 60 minutos, pues si la divisón es exacta, tenemos horas completas, de lo contrario, ya inicio otra hora y se debe añadir una hora más al conteo de horas adicionales.

Reparación del Código Original

```
...
else:
    horas_adicionales = math.ceil((minutos - 60) / 60) # Modificación
    total = 20.0 + (horas_adicionales * 15.0)
...
```

Para reparar el código correctamente conforme al modelo, debemos tomar el techo del resultado de la división para contemplar verdaderamente el número de horas adicionales.

Casos Frontera: 4, 5, 6, 7, 8 y 9

Implementación de la Prueba en Python

Prueba encargada de verificar el comportamiento del sistema en los casos frontera 4, 5, 6, 7, 8 y 9.

```
# Arrange
@pytest.mark.parametrize(
    "minutos, esperado",
    [
                      # | Caso | Descripción                              |
        (15, 0.00),   # |  4   | Último valor entre el rango de 1-15.     |
        (16, 20.00),  # |  5   | Primer valor entre el rango de 16-60.    |
        (60, 20.00),  # |  6   | Último valor entre el rango de 16-60.    |
        (61, 35.00),  # |  7   | Una hora completa y una hora iniciada.   |
        (120, 35.00), # |  8   | Dos horas completas sin hora iniciada.   |
        (121, 50.00), # |  9   | Dos horas completas y una hora iniciada. |
    ],
)
def test_cliente_Normal_con_Boleto_Casos_Frontera(sistema, minutos, esperado):
    """
    Verifica el cobro calculado para un cliente "normal" con boleto dado un cierto
tiempo,
    en escencia cuando hay un cambio entre los límites de rangos de tiempo.
    """

    # Act
    resultado = sistema.calcular_total(minutos, "normal", False)

    # Assert
    assert resultado == esperado
```

Resultado de la Prueba

Los casos 5, 6, 7, 8 y 9 pasaron la prueba; sin embargo el caso 4 la falló. Dado que el planteamiento de las pruebas esta basado en una interpretación correcta del modelo, analizamos el Código original.

Código Original

```
...
elif minutos < 15:
    total = 0.0
...
```

Al revisar el Código original, notemos que el cálculo realizado para conocer el cobro del estacionamiento en 15 minutos es incorrecto, ya que no se toma en cuenta el minuto 15 para cobrar la cantidad correcta de $0.00.

Reparación del Código Original

```
...
elif minutos <= 15: # Modificación
    total = 0.0
...
```

Para reparar el código correctamente conforme al modelo, debemos tomar en cuenta al minuto 15 dentro del rango correspondiente y cobrar $0.00.

Casos Entrada Inválida: 10, 11 y 15

Implementación de la Prueba en Python

Prueba encargada de verificar el comportamiento del sistema en los casos de entrada inválida 10, 11 y 15, este último propuesto por el equipo.

Antes de escribir la prueba, recordemos que tener excepciones muy generales para los "errores" que maneje el programa, indican una mala práctica de programación, ya que dificulta la documentación y posterior mantenimiento del programa.

Por lo anterior, buscamos las excepciones correspondientes que debería lanzar el programa dados los casos de entradas inválidas que hemos establecido, pero dado que no las encontramos, procedemos a definir excepciones personalizadas que documenten el caso correpondiente:

- **MinutosInvalidos:** Lanzada cuando no se cumple **R1** y **R10**.
- **ClienteInvalido:** Lanzada cuando no se cumple **R2**.

Con ello, definimos las pruebas con las nuevas excepciones que esperamos ver.

```
...
from MinutosInvalidos import MinutosInvalidos
from ClienteInvalido import ClienteInvalido
...
```

```
    # Test de Entradas inválidas.

    # Arrange
    @pytest.mark.parametrize(
        "minutos",
        [
                    # | Caso | Descripción                   |
            -1,     # | 10   | -1 min no están definidos.    |
            15.5,   # | 15   | 15.5 min no están definidos.  |
        ],
    )
    def test_Minutos_Invalidos(sistema, minutos):
        """
        Verifica que el programa rechaze los Minutos Inválidos establecidos
        en la R1.
        """
        # Siguiendo la R10, verifcamos que rechaze los minutos negativos.
        # Act and Assert
        with pytest.raises(MinutosInvalidos):
            sistema.calcular_total(minutos, "normal", False)

    def test_Cliente_Invalido(sistema):
        """
        Verifica que el programa rechaze los clientes que no están
        definidos en la R2. Siguiendo el procedimiento de R11.
        """
        # | Caso | Descripción                              |
        # | 10   | Cliente "premium" no está bien definido.   |

        # Arrange, Act and Assert
        with pytest.raises(ClienteInvalido):
            sistema.calcular_total(5, "premium", False)
    ...
```

Resultado de la Prueba

Los casos 10, 11 y 15 fallaron la prueba; dado que el planteamiento de las pruebas esta basado en una interpretación correcta del modelo, analizamos el Código original.

Código Original

```
    ...
    if minutos < 0:
        raise ValueError("Los minutos no pueden ser negativos")

    if tipo_cliente not in ("normal", "frecuente"):
       raise ValueError("Tipo de cliente no válido")
    ...
```

Al revisar el Código original, notamos que para el caso 10 y 11 no se lanzan las excepciones que esperabamos porque la implementación original reporta las entradas inválidas con la excepción ValueError, pero como mencionamos anteriormente, esta es una mala práctica de programación. Por otro lado, en el caso 15, la implementación original no considera a los decimales como valor de entrada inválida para los minutos, cuando debería hacerlo por el modelo establecido.

Reparación del Código Original

```
...
if minutos < 0:
    raise MinutosInvalidos("Los minutos no pueden ser negativos") # Modificacion

if isinstance(minutos, float): # Modificación
    raise MinutosInvalidos("Los minutos no pueden ser en decimal")

if tipo_cliente not in ("normal", "frecuente"):
    raise ClienteInvalido("Tipo de cliente no válido") # Modificacion
...
```

Para reparar el código correctamente conforme al modelo, modificamos las excepciones que se lanzan cuando hay valores de entrada inválidos, para los minutos la excepción MinutosInvalidos y para los tipos de cliente ClienteInvalido, además, agregamos el caso donde consideramos a los decimales como valor inválido para los minutos.

Casos Interacción entre reglas: 12 y 13

Implementación de la Prueba en Python

Prueba que verifica el comportamiento del sistema en los casos de interacción entre reglas 12 y 13.

```
...
# Arrange
@pytest.mark.parametrize(
    "boleto, esperado",
    [
                          # | Caso | Descripción                                      |
        (False, 288.00),  # | 12   | Cliente "frecuente" con boleto con 1250 min. |
        (True, 300.00),   # | 13   | Cliente "frecuente" sin boleto con 1250 min. |
    ],
)

def test_cliente_Frecuente_Casos_Interaccion_entre_reglas(sistema, boleto, esperado):
    """
    Verifica que el programa calcule correctamente el costo del estacionamiento
    dado que el cliente es frecuente, pase 1250 minutos y tenga o no boleto.
    """
    # Act
    resultado = sistema.calcular_total(1250, "frecuente", boleto)
```

```
        # Assert
        assert resultado == esperado
    ...
```

Resultado de la Prueba

El caso 12 pasó la prueba, pero el 13 la falló; dado que el planteamiento de las pruebas esta basado en una interpretación correcta del modelo, analizamos el Código original.

Código Original

```
    ...
    if boleto_perdido:
        total = self.TARIFA_BOLETO_PERDIDO
    ...
    else:
        horas_adicionales = math.ceil((minutos - 60) / 60)
        total = 20.0 + (horas_adicionales * 15.0)

    if tipo_cliente == "frecuente":
        total *= 0.90

    return round(total, 2)
    ...
```

Al revisar el Código original, notamos que el flujo para asignarle el costo a un cliente que perdió su boleto está incorrecto, pues si bien asigna la tarifa de $300.00, no termina el cálculo y prosigue a continuar evaluando casos, como el tipo de cliente, cuando en realidad la R3 prevalece sobre la R8.

Reparación del Código Original

```
    ...
    if boleto_perdido:
        return self.TARIFA_BOLETO_PERDIDO # Modificación.
    ...
    else:
        horas_adicionales = math.ceil((minutos - 60) / 60)
        total = 20.0 + (horas_adicionales * 15.0)

    if tipo_cliente == "frecuente":
        total *= 0.90

    return round(total, 2)
    ...
```

Para reparar el código correctamente conforme al modelo, modificamos el flujo; si el cliente perdió su boleto, simplemente retornamos el costo del estacionamiento de $300.00 sin pasar por los otros casos.

Caso Números decimales en el resultado: 14

Implementación de la Prueba en Python

Prueba que verifica el comportamiento del sistema en el caso 14 de números decimales en el resultado.

```
...
# Arrange
@pytest.mark.parametrize(
    "minutos, esperado",
    [
                      # | Caso | Descripción                                        |
        (61, 31.50),  # | 14   | Cliente "frecuente" con boleto paso 61 minutos.  |
    ],
)
def test_cliente_Frecuente_Casos_Interaccion_entre_reglas(sistema, minutos, esperado):
    """
    Verifica que el programa calcule correctamente el costo del estacionamiento
    dado que el cliente es frecuente y tenga boleto.
    """
    # Act
    resultado = sistema.calcular_total(minutos, "frecuente", False)

    # Assert
    assert resultado == pytest.approx(esperado)
    ...
```

En este caso, podemos ver que el resultado proviene de operaciones con float, y estas pueden aparecer con pequeñas diferencias de precisión por el manejo de este tipo de dato en el lenguaje, por ello, ocupamos pytest.approx para comparar valores con cierta tolerancia que definimos como correctos. En los anteriores casos no es necesario porque obtenemos resultados enteros que si pueden compararse exactamente con el operador "==" sin utilizar pytest.approx.

Resultado de la Prueba

El caso 14 pasó la prueba; dado que el planteamiento de las pruebas esta basado en una interpretación correcta del modelo, determinamos que el Código original está bien planteado para este caso.

Código Original

```
...
def calcular_total(self, minutos: int, tipo_cliente: str = "normal",
                   boleto_perdido: bool = False) -> float:
    ...
    if boleto_perdido:
        return self.TARIFA_BOLETO_PERDIDO
    ...
    else:
        horas_adicionales = math.ceil((minutos - 60) / 60)
        total = 20.0 + (horas_adicionales * 15.0)
```

```
        if tipo_cliente == "frecuente":
            total *= 0.90

        return round(total, 2)
    ...
```

Al revisar el Código original y compararlo con el modelo del sistema, round funciona bien para determinar el costo de la tarifa en centavos. Pero en este caso, se vuelve un poco redundante al trabajar con tarifas que solo tengan como decimal a 0.0 o 0.5 y estos representarse de forma exacta en binario; sin embargo, en caso de que llegasen a cambiar los precios posteriormente, entonces sería de más utilidad dependiendo de como se establezca el redondeo de tarifas en el nuevo modelo.

Preguntas de análisis

1. ¿Qué diferencia hay entre un caso normal y un caso frontera?

Un caso normal toma valor que se encuentra dentro un rango sin llegar a los valores extremos, es decir, donde se mantiene un mismo comportamiento.

En cambio, un caso frontera busca ver como se comportan los valores en los extremos de dos rangos, donde se puede apreciar hasta donde se mantiene un comportamiento y luego donde empieza otro comportamiento distinto.

2. ¿Cuál de sus pruebas considera más importante y por qué?

En general las pruebas sirven para comprobar el funcionamiento de un sistema alineado al modelo del mismo, por ello el conjunto de pruebas es crucial, pues cada una aporta a la dismunución de bugs en la implementación sistema y lo acerca más a la resolución del problema original con un modelo establecido.

Sin embargo, en caso de elegir, notemos que la prueba de interacción entre reglas es crucial, pues aparte de corroborar casos como el 12 o el 13, que son de interacción entre reglas, implicitamente sigue corroborando que siga funcionando el sistema con un caso normal, o incluso si modificamos los minutos, que el sistema siga funcionando con casos frontera.

3. ¿Encontró algún comportamiento de la implementación que no coincida con el modelo?

Bastantes, como el no cobrar correctamente las horas inciadas pasada la primera hora, el permitir decimales en los minutos, el no cobrar correctamente si un cliente pierde su boleto, el no lanzar excepciones especializadas para informar las entradas inválidas y no cobrar correctamente si un cliente pasaba 15 minutos en el estacionamiento.

4. ¿Una suite con 100 % de pruebas aprobadas demuestra que el programa es correcto? Explique.

No necesariamente, por ejemplo, si las pruebas estan planteadas de acuerdo al código y no al modelo del sistema que desamos implementar, obtendremos una suite que solo prueba lo que ya esta hecho pero no el comportamiento que espereamos.

Además, las pruebas sirven para contemplar los casos en general que podemos encontrar en un sistema, pero si no se tiene cuidado o se plantea mal el razonamiento del modelo, en futuro nos encontremos con un nuevo problema que no se tuvo en cuenta al momento de hacer el mismo modelo o las pruebas.

5. Si una IA generara automáticamente 50 pruebas, ¿qué tendría que revisar una persona antes de confiar en ellas?

Tendría que ver si las pruebas tienen sentido, es decir, que las pruebas esten fundamentadas en el modelo del sistema que deseamos implementar, y a priori el modelo este planteado para resolver correctamente el problema original.

Además, la persona debe evitar pruebas innecesarias o repetidas, por ejemplo, no sirve mucho tener 50 pruebas que solo tomen en cuenta casos normales, o solo fronteras, etc.